

Funções de Alta Ordem

COMP0496 – Programação A

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
`andreyoshiaki@dcomp.ufs.br`

Duas funções quase idênticas — só o que fazem com cada elemento muda:

```
1 def dobrar_lista(nums):  
2     return [x * 2 for x in nums]  
3  
4 def triplicar_lista(nums):  
5     return [x * 3 for x in nums]
```

E se precisarmos de `quadruplicar_lista`, `elevao_quadado_lista`...

Solução: parametrizar o *comportamento* — passar a operação como argumento.

Função de Alta Ordem

Função que recebe outra função como argumento **e/ou** retorna uma função como resultado.

```
1 def aplicar(f, lista):
2     return [f(x) for x in lista]
3
4 def dobrar(x): return x * 2
5 def triplicar(x): return x * 3
6
7 nums = [1, 2, 3, 4, 5]
8 print(aplicar(dobrar, nums))      # [2, 4, 6, 8, 10]
9 print(aplicar(triplicar, nums))  # [3, 6, 9, 12, 15]
```

aplicar não sabe *o que faz* — apenas *aplica* o comportamento recebido.

Uma função pode *retornar* outra função, capturando variáveis do escopo externo:

```
1  def multiplicador(n):
2      def multiplicar(x):
3          return x * n    # captura 'n' do escopo externo
4      return multiplicar
5
6  dobrar    = multiplicador(2)
7  triplicar = multiplicador(3)
8
9  print(dobrar(5))      # 10
10 print(triplicar(5))   # 15
11 print(dobrar(7))      # 14
```

`dobrar` e `triplicar` são funções criadas dinamicamente — cada uma lembra o valor de `n`.

Para funções simples de uma expressão, Python oferece a sintaxe `lambda`:

`lambda` *parâmetros*: *expressão*

```
1  dobrar = lambda x: x * 2
2  print(dobrar(5))      # 10
3
4  soma = lambda a, b: a + b
5  print(soma(3, 4))     # 7
6
7  # Inline, sem nome:
8  print((lambda x: x ** 2)(6)) # 36
```

Limitação: apenas uma expressão — sem `if/for` múltiplos ou `return` explícito.

O argumento `key=` recebe uma função que define o critério de ordenação:

```
1  alunos = [  
2      {"nome": "Carlos", "nota": 7.5},  
3      {"nome": "Ana",    "nota": 9.0},  
4      {"nome": "Bob",    "nota": 6.0},  
5  ]  
6  
7  # Ordenar por nota crescente  
8  por_nota = sorted(alunos, key=lambda a: a["nota"])  
9  for a in por_nota:  
10     print(a["nome"], a["nota"])  
11  # Bob 6.0 | Carlos 7.5 | Ana 9.0  
12  
13  # Ordenar por nota decrescente  
14  ranking = sorted(alunos, key=lambda a: a["nota"],  
15                  reverse=True)
```

Converter temperaturas em Celsius para Fahrenheit usando aplicar:

```
1  def aplicar(f, lista):
2      return [f(x) for x in lista]
3
4  celsius_para_f = lambda c: c * 9/5 + 32
5
6  celsius = [0, 20, 37, 100]
7  fahrenheit = aplicar(celsius_para_f, celsius)
8  print(fahrenheit)  # [32.0, 68.0, 98.6, 212.0]
9
10 # Compor: converter E arredondar
11 arredondar = lambda x: round(x, 1)
12 resultado = aplicar(arredondar, fahrenheit)
13 print(resultado)  # [32.0, 68.0, 98.6, 212.0]
```

- Em Python, **funções são valores** — podem ser passadas e retornadas como qualquer objeto.
- **Função de alta ordem:** recebe e/ou retorna funções, parametrizando comportamentos.
- **Closure:** função interna captura variáveis do escopo externo.
- **Lambda** `lambda x: expr` cria funções anônimas simples de uma expressão.